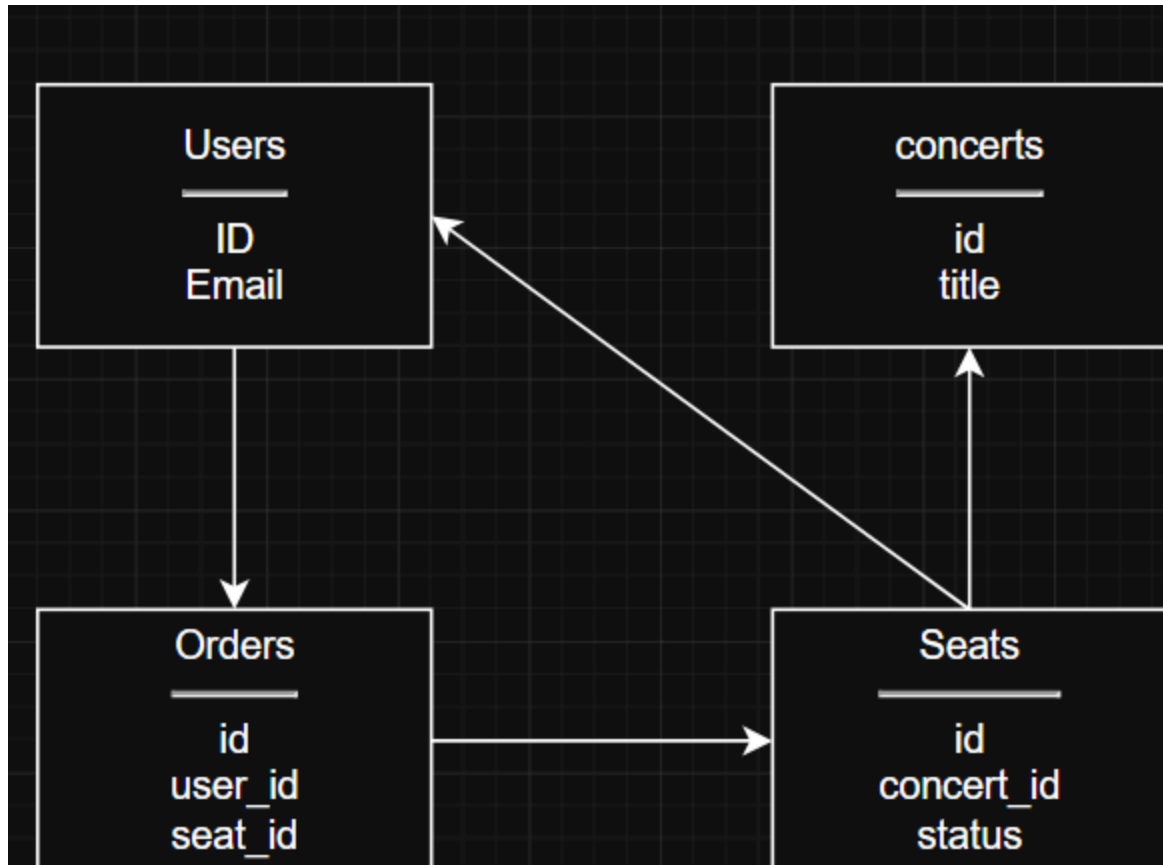


Сервіс бронювання квитків

1. Опис архітектури системи

- **Web Server:** FastAPI
- **База даних:** PostgreSQL.
- **Кешування:** Redis

2. Схема бази даних



Детальний опис таблиць та індексів:

1. users (Користувачі)

- id: INTEGER (Primary Key, Autoincrement)
- email: VARCHAR(255) (Unique Index — для швидкої авторизації)
- password_hash: VARCHAR(255) (Хеш пароля)
- created_at: TIMESTAMP (Час реєстрації)

2. concerts (Заходи)

- id: INTEGER (Primary Key)

- title: VARCHAR(255) (Індекс для пошуку)
- artist: VARCHAR(255)
- genre: VARCHAR(50) (Індекс для швидкої фільтрації в каталозі)
- location: TEXT
- date_time: TIMESTAMP WITHOUT TIME ZONE
- base_price: NUMERIC(10, 2)

3. seats (Місця в залі)

- id: INTEGER (Primary Key)
- concert_id: INTEGER (Foreign Key -> concerts.id, ON DELETE CASCADE)
- row_number: INTEGER
- seat_number: INTEGER
- status: VARCHAR(20) (Значення: FREE, BOOKED, SOLD)
- reserved_by_user_id: INTEGER (Foreign Key -> users.id, Nullable)
- reserved_until: TIMESTAMP WITHOUT TIME ZONE (Час закінчення 10-хвилинного резерву)
- Індокси: Складений унікальний індекс uq_concert_seat (concert_id, row_number, seat_number) для унеможливлення створення дублікатів місць.

4. orders (Історія покупок)

- id: INTEGER (Primary Key)
- user_id: INTEGER (Foreign Key -> users.id)
- seat_id: INTEGER (Foreign Key -> seats.id)
- amount_paid: NUMERIC(10, 2)
- purchased_at: TIMESTAMP WITHOUT TIME ZONE

3. Обґрунтування ключових технічних рішень

- **Вибір PostgreSQL:** Оскільки додаток оперує фінансовими транзакціями та кінцевою кількістю місць (квитків), важливо забезпечити дотримання вимог **ACID**, він гарантує повну ізоляцію транзакцій.
- **Підхід до багатопотоковості:** Було обране **Песимістичне блокування**. Під час перевірки статусу місця рядок миттєво блокується поточною транзакцією. Усі інші конкурентні запити чекають звільнення блокування або отримують відмову.
- **Redis:**
 - *Що кешується:* Повний список доступних концертів.
 - *Прибирання кешу:* Відбувається кожну хвилину для оновлення дат

4. Інструкція із запуску застосунку у хмарі

Програму повністю укомплектованно та розгорнуто в хмарі на сервері Azure

Запуск системи:

1. Підключіться до вашої віддаленої віртуальної машини через SSH:
ssh -i ключ username@<your_public_ip> (в нашому випадку це був
ssh -i A:\download\ticket-booking_key.pem azureuser@172.201.98.124
2. Клонуємо репозиторій з Git
git clone https://github.com/Codlvy/ticket.git

```
cd ticket
```

3. Запускаємо Docker Compose:
`docker-compose up --build -d`
4. Про всяк випадок перевіряємо статус контейнерів
`docker-compose ps`

5. Запуск навантажувального тесту на основі k6 який симулює 100 покупок на одне місце (якщо зазначене місце заброньоване в скрипті потрібно змінити id на інше число)

```
k6 run testing.js
```

Результати тестування:

```
execution: local
  script: testing
  output: -

scenarios: (100.00%) 1 scenario, 100 max VUs, 45s max duration (incl. graceful stop):
  * race_condition_test: 1 iterations for each of 100 VUs (maxDuration: 15s, gracefulStop: 30s)

TOTAL RESULTS

checks_total.....: 100      55.786708/s
checks_succeeded....: 100.00% 100 out of 100
checks_failed.....: 0.00%   0 out of 100

✔ Cratyc 303 a60 409

HTTP
http_req_duration.....: avg=1.52s min=92ms   med=1.62s max=1.68s p(90)=1.66s p(95)=1.67s
  { expected_response:true }....: avg=92ms min=92ms   med=92ms max=92ms p(90)=92ms p(95)=92ms
http_req_failed.....: 99.00% 99 out of 100
http_reqs.....: 100      55.786708/s

EXECUTION
iteration_duration.....: avg=1.62s min=181.91ms med=1.71s max=1.78s p(90)=1.76s p(95)=1.77s
iterations.....: 100      55.786708/s
vus.....: 96      min=96      max=96
vus_max.....: 100      min=100      max=100

NETWORK
data_received.....: 18 kB  9.8 kB/s
data_sent.....: 18 kB  10 kB/s

running (01.8s), 000/100 VUs, 100 complete and 0 interrupted iterations
race_condition_test ✔ [=====] 100 VUs  01.8s/15s  100/100 iters, 1 per VU
```

Тест підтверджує, що система опрацювала запити коректно, тільки 1 запит пройшов, всі інші були відхиленні

SQL-запит для перевірки стану броні в БД:

```
docker exec -it ticket_postgres psql -U ticket_user -d ticket_service_db -c "SELECT id, status, reserved_by_user_id, reserved_until FROM seats WHERE id = 1;"
```

Результат:

id	status	reserved_by_user_id	reserved_until
1	BOOKED	999	2026-06-25 11:15:14.49423
(1 row)			

6. Публічна IP-адреса застосунку

Застосунок розгорнуто, налаштовано відкриття портів та повністю перевірено доступність із зовнішньої мережі за такою адресою:

- **Публічний URL сайту:** <http://172.201.98.124:8000/concerts>
- **Документація API (Swagger):** <http://172.201.98.124:8000/docs>